Pytorch 입문 부트캠프

3. Pytorch Exercises (4) Time Series Model

목차

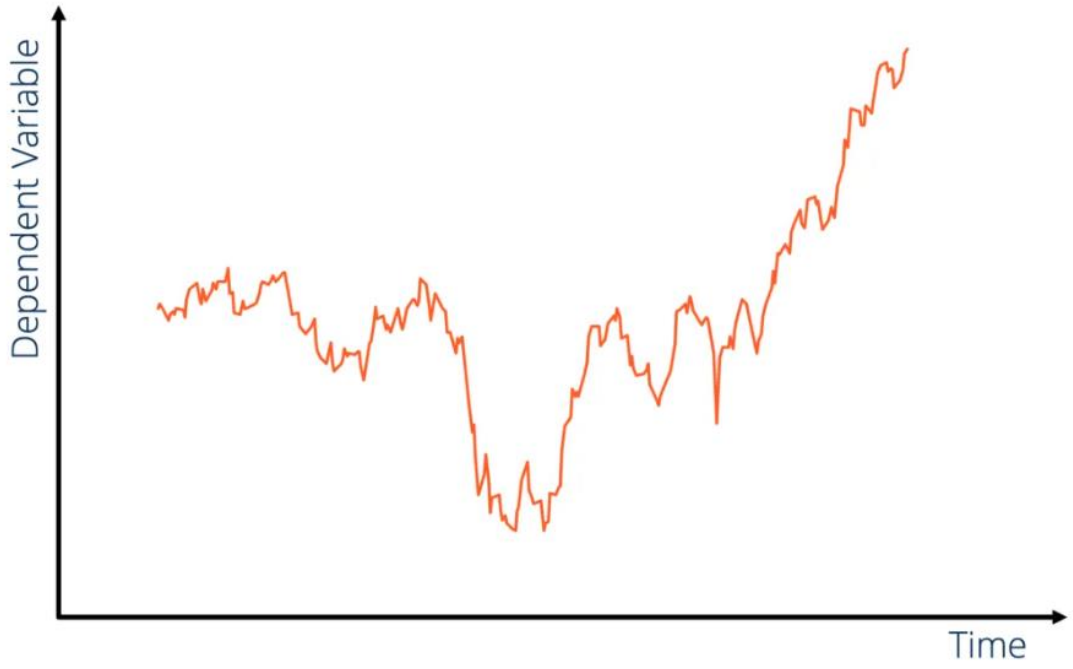
1

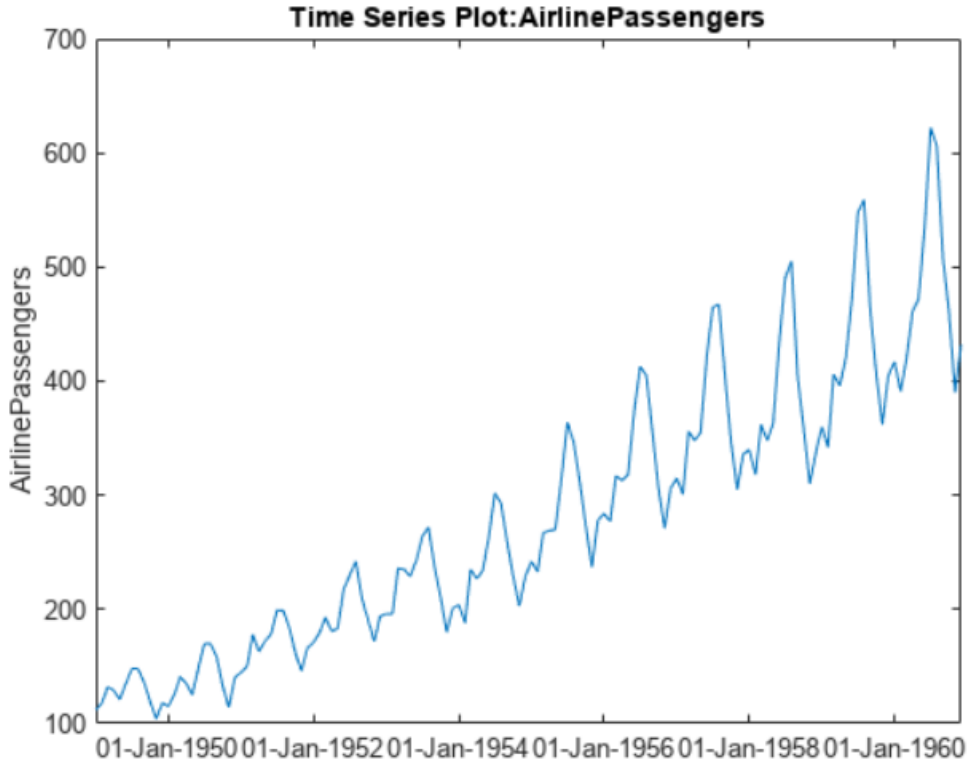
What is Time Series?

Part 1

What is Time Series?

- Sequence Data / Time Series Data ?
  - Sequence Data : 순서가 의미가 있으며 순서가 달라질 경우 의미가 손상되는 데이터, 시간적인 의미가 있다면 temporal data라고도 함
  - Time Series (시계열 데이터) : 일정 시간 간격으로 기록된 데이터
  - 과거 패턴을 기반으로 미래를 예측하거나 (Forecasting) 과거 특징적인 부분들을 분석

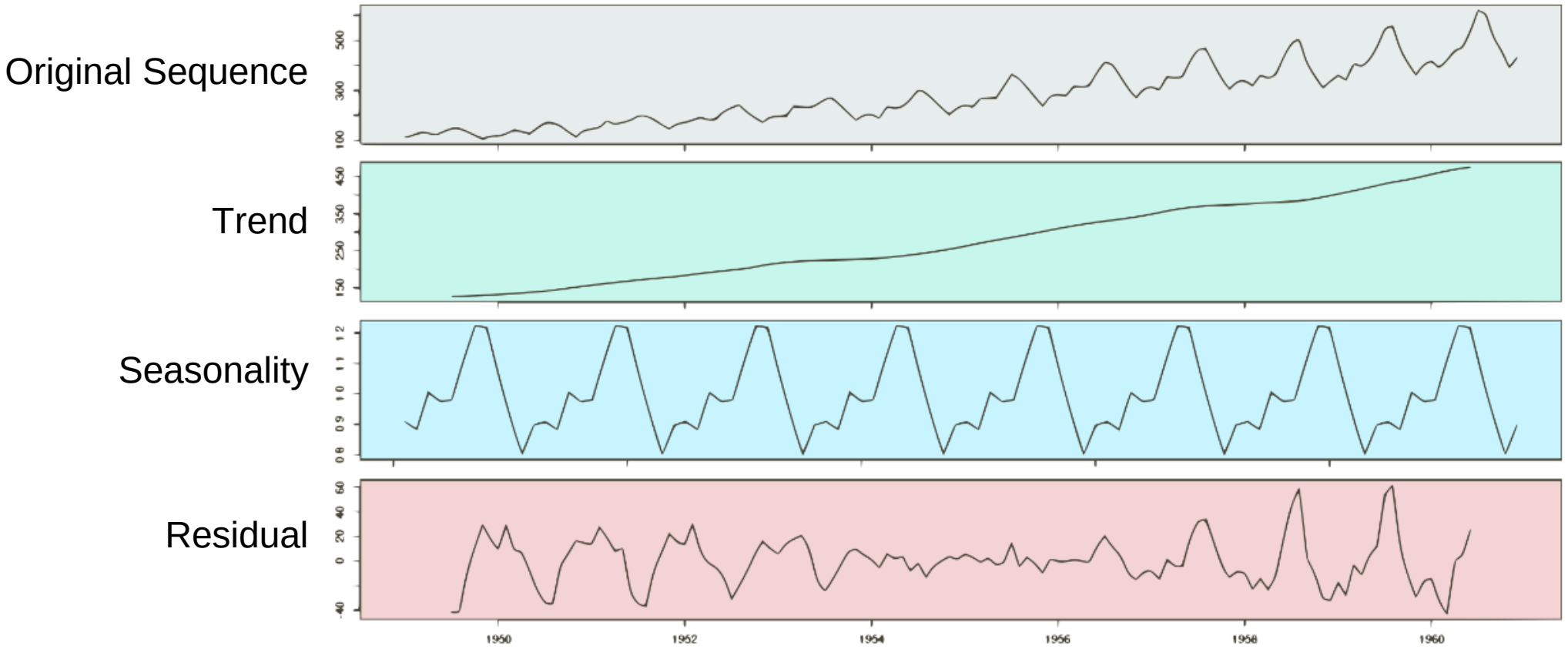




Part 1

What is Time Series?

- 통계적 분석 방법론 (딥러닝 도입 전 기존 시계열 모델)
  - 원본 시계열에서 Trend, Seasonality를 제거하고 남은 Residual 을 모델링
  - Residual은 Stationarity 라는 특징을 기반으로 모델링 (AR, MA, ARIMA, SARIMA 등)
  - 예측은 반대로 Residual 을 토대로 만든 모델의 예측 값에 Trend와 Seasonality를 더해 계산

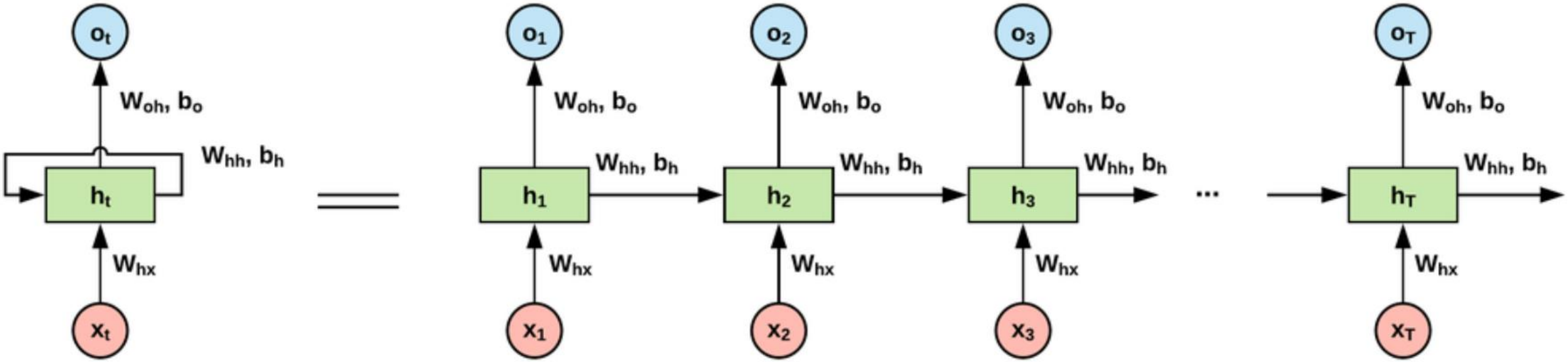


2

Recurrent Neural Network, LSTM

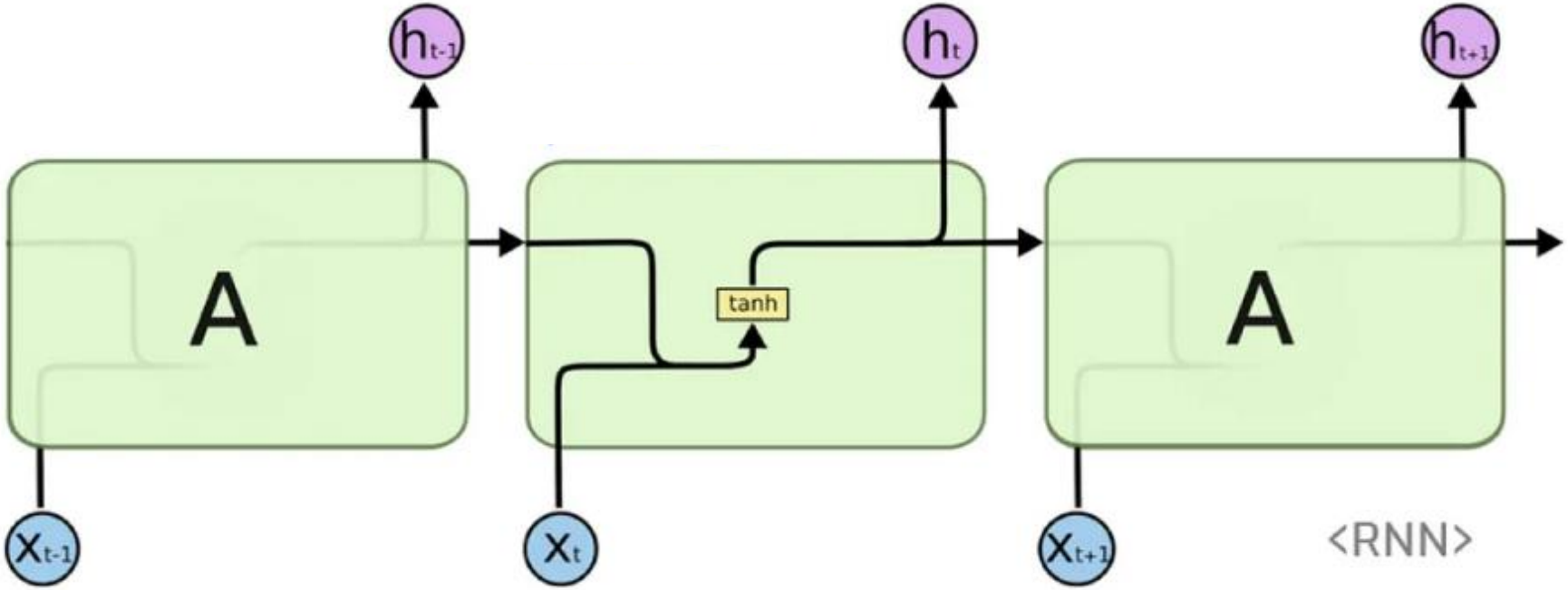
Recurrent Neural Network, LSTM

- Recurrent Neural Network?
  - Sequence Data 여러 개를 input으로 받아 각 input마다 output을 내면서, 동시에 중간 계산 결과인 hidden state를 다음 hidden state 계산 때 합쳐서 계산하는 구조
  - 위의 과정을 sequence size만큼 반복



Recurrent Neural Network, LSTM

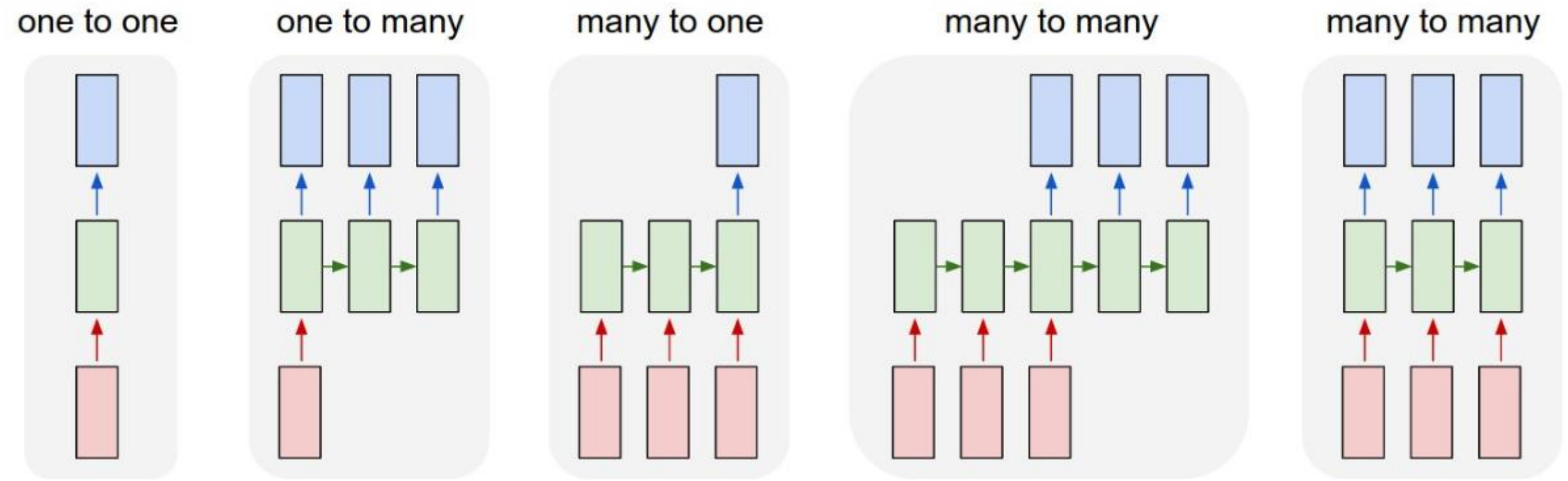
- RNN layer



- $h_t = \tanh\left(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{ih}\right)$
- Input sequence가 1에서 T까지 들어올 때 각 input마다 개별적인 W_{ih}, b_{ih} 을 통해 계산하고, 그 결과를 이전 h_{t-1} 을 W_{hh}, b_{hh}를 통해 계산한 결과와 합쳐서 tanh 통과 → 다음 h_t

Recurrent Neural Network, LSTM

- RNN의 종류



Text Classification
감정 분석

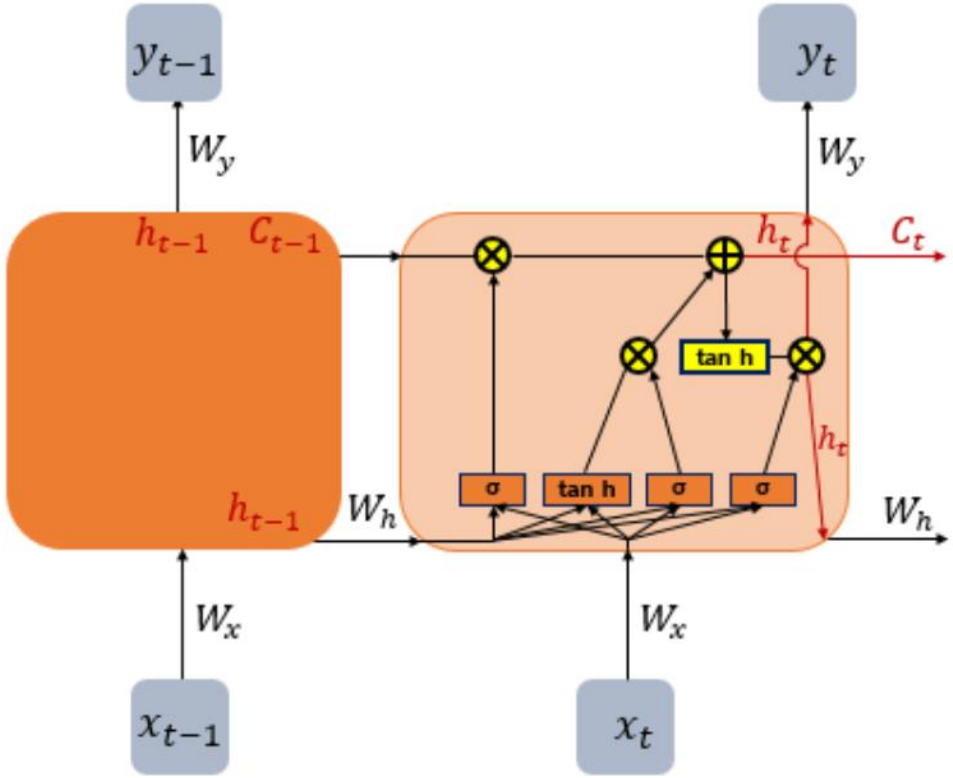
요약, 생성

번역

Recurrent Neural Network, LSTM

- RNN의 단점
  - 장기 의존성 문제(the problem of Long-Term Dependencies) : input sequence가 길어지면 초기값의 정보량이 뒤쪽으로 충분히 전달되지 않는 RNN의 문제점
  - Gradient Vanishing, Exploding

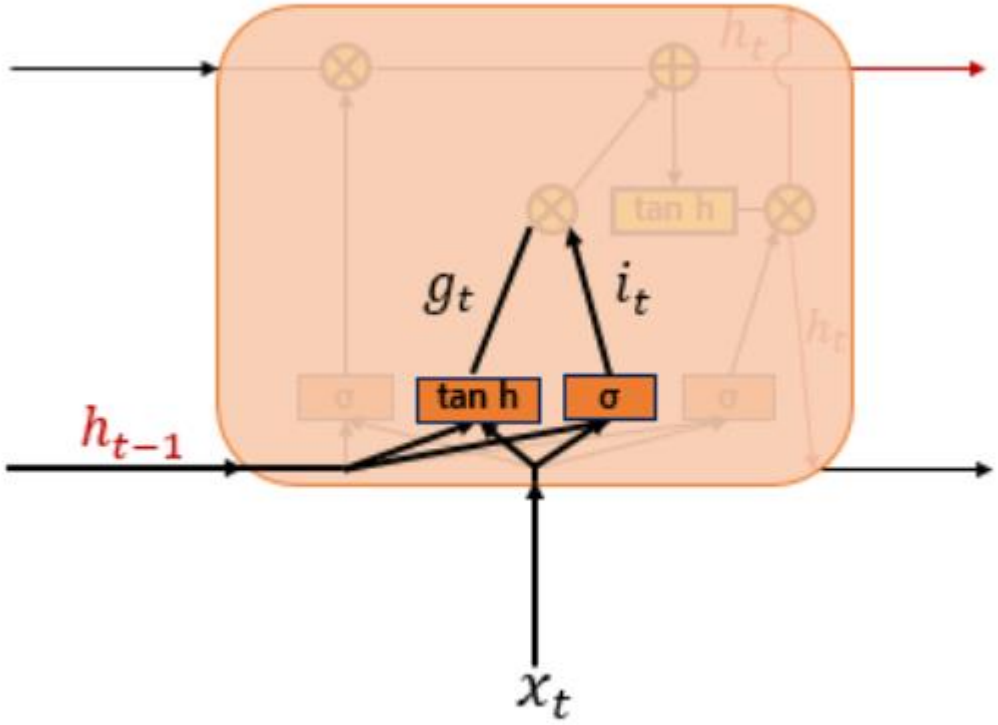
- LSTM (Long Short-Term Memory)
  - RNN의 변형
  - Cell State의 정의
  - 기존의 hidden state를 셀, 입력 게이트, 출력 게이트, 망각 게이트로 세분화 하여 각 게이트 마다 출력의 정도를 조절하여 사용
  - 좀 더 복잡한 구조를 통해 RNN과 비교하여 긴 시퀀스의 입력을 처리하는데 탁월한 성능



Recurrent Neural Network, LSTM

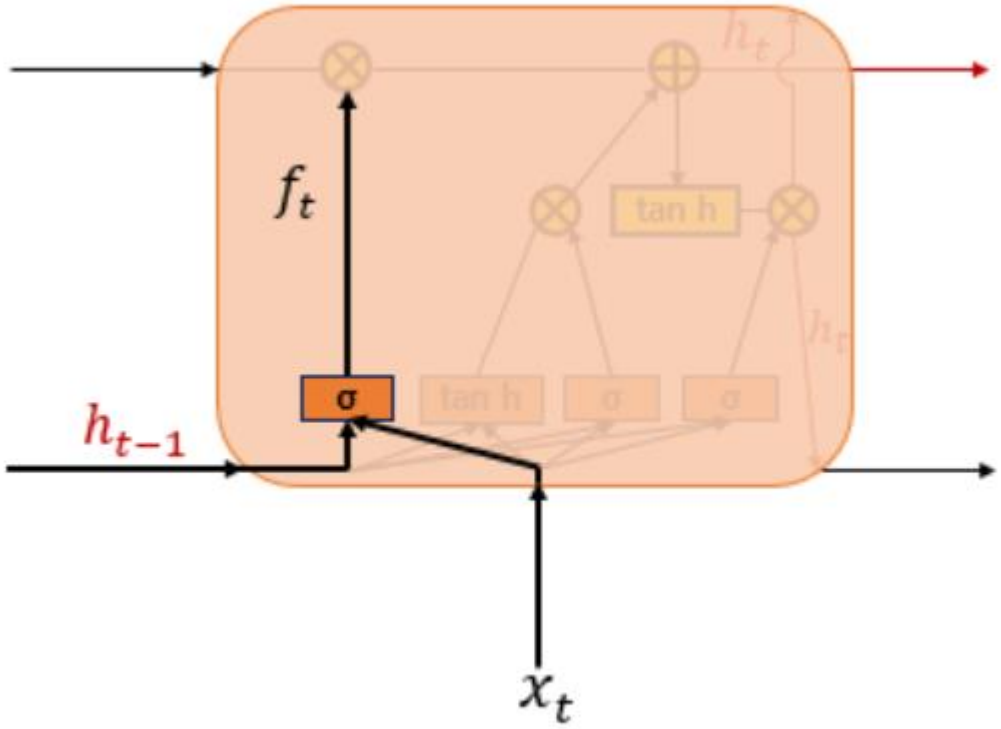
- LSTM의 구조

(1) 입력 게이트



$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$$
$$g_t = tanh(W_{xg}x_t + W_{hg}h_{t-1} + b_g)$$

(2) 삭제 게이트

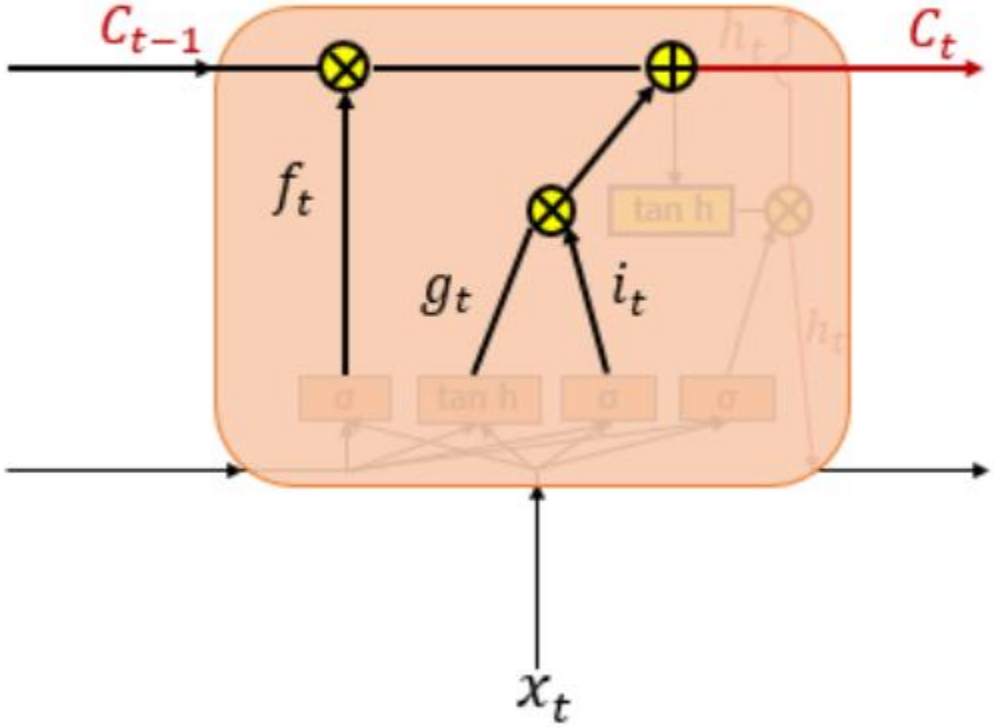


$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$$

Recurrent Neural Network, LSTM

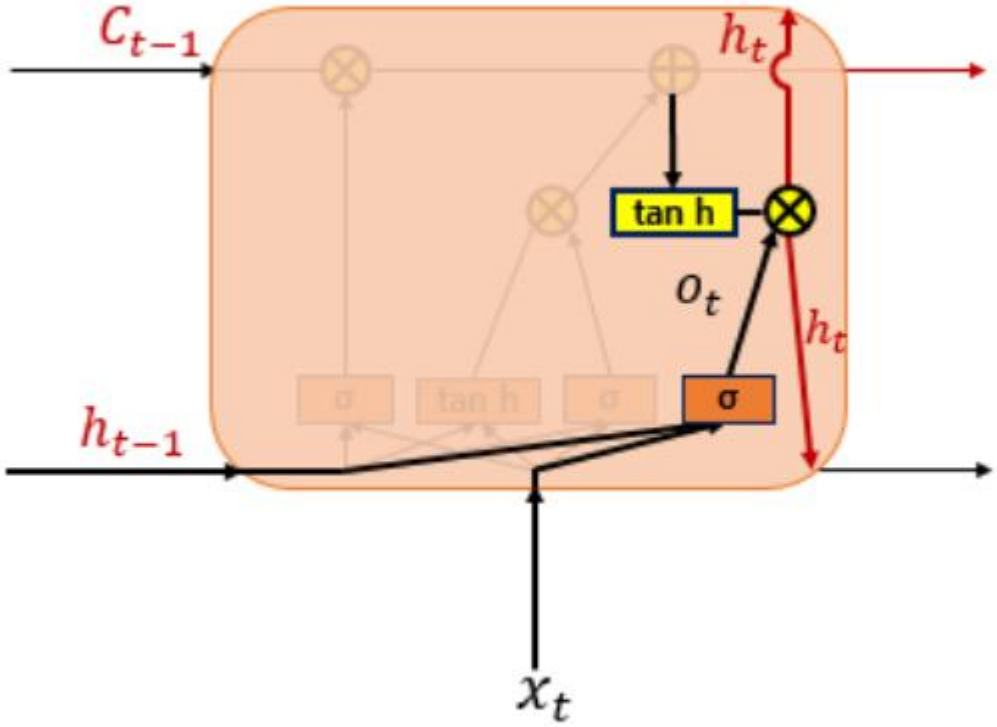
- LSTM의 구조

(3) 셀 상태



$$C_t = f_t \circ C_{t-1} + i_t \circ g_t$$

(4) 출력 게이트와 은닉 상태



$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$$
$$h_t = o_t \circ tanh(c_t)$$

3

RNN, LSTM in Pytorch

RNN, LSTM in Pytorch

- RNN Layer
  - Input Tensor shape

```
# input tensor 선언
input = torch.ones(5, 3) # seq_size, input_dim
# input = torch.ones(11, 5, 3) # batch_size, seq_size, input_dim
```

  - RNN Layer 정의

```
# RNN Layer 선언
rnn_layer = nn.RNN(input_size=3, hidden_size=10)
```

  - Output shape

```
# output 출력
output, hidden = rnn_layer(input)
print(output.shape)
print(hidden.shape)
```

```
torch.Size([5, 10])
torch.Size([1, 10])
```

RNN, LSTM in Pytorch

- LSTM Layer
  - Input Tensor shape

```
# input tensor 정의
input = torch.ones(7, 3) # seq_size, input_dim
# input = torch.ones(11, 7, 3) # batch_size, seq_size, input_dim
```

  - LSTM Layer 정의

```
# layer 정의
lstm_layer = nn.LSTM(input_size=3, hidden_size=10)
```

  - Output shape

```
# output 출력
output, (hidden, cell) = lstm_layer(input)
print(output.shape)
print(hidden.shape)
print(cell.shape)
```

```
torch.Size([7, 10])
torch.Size([1, 10])
torch.Size([1, 10])
```

RNN, LSTM in Pytorch

- 데이터셋 구성 (네이버 주식 웹 크로링을 통해 데이터 가져오기)

```
from bs4 import BeautifulSoup
import requests

# 웹 크롤링
code = '005930' # 삼성전자 종목코드
url = 'https://finance.naver.com/item/sise_day.naver?code={}'.format(code)
header = {'User-agent':'Mozilla/5.0'}
req = requests.get(url, headers=header)
html = BeautifulSoup(req.text, "lxml")

pgrr = html.find('td', class_='pgRR')
s = pgrr.a['href'].split('=')
last_page = s[-1]

# dataset 구성
df = None

for page in tqdm(range(1, 100)):
    req = requests.get('{}&page={}'.format(url, page), headers=header)
    df = pd.concat([df, pd.read_html(req.text, encoding='euc-kr')[0]], ignore_index=True)
```

RNN, LSTM in Pytorch

- 데이터셋 구성 (전처리)

```
# df 전처리
df.dropna(inplace=True)

# 컬럼명 변경
df.columns = ['Date', 'Close', 'Gap', 'Open', 'High', 'Low', 'Amount']

# 불필요 컬럼 제거
df.drop(['Gap'], axis=1, inplace=True)

# 시간순 정렬
df.sort_values(['Date'], inplace=True)
df.reset_index(drop=True, inplace=True)
```

Part 3

RNN, LSTM in Pytorch

- 데이터셋 구성 (train/test 분리, 데이터 스케일링)

```
# train / test 분리 (train 만 이용해서 학습 및 추론, test는 모델에 input으로 사용 X(미래 값이므로))
train_df = df.loc[df.Date < '2024.04.03'] # 여기까지만 사용
test_df = df.loc[df.Date >= '2024.04.03'] # 모델 학습이 끝난 뒤에 성능 평가용
print(train_df.tail())
print(test_df)

train_df.drop(['Date'], axis=1, inplace=True)

# scaling
scaler = StandardScaler()
scaler.fit(train_df)
s_train_df = scaler.transform(train_df)
s_test_df = scaler.transform(test_df.drop(['Date'], axis=1))
```

RNN, LSTM in Pytorch

- 데이터셋 구성 (시계열 구조로 변경 : input_window, output_window)

```
# Define the window sizes
input_window = 30
output_window = 5

# Create the dataset
data = torch.tensor(s_train_df, dtype=torch.float32)
target = torch.tensor(s_train_df.T[0], dtype=torch.float32)

# Create the input and output sequences
X, Y = [], []
for i in range(len(data) - input_window - output_window + 1):
    X.append(data[i:i+input_window])
    Y.append(target[i+input_window:i+input_window+output_window])

# Create the TensorDataset
dataset = TensorDataset(torch.stack(X), torch.stack(Y))
print(dataset[0][0].shape) # seq_size(30일), input_size(close, Open, high, low, amount)
print(dataset[0][1].shape) # output_size(30일치 Close)
```

Part 3

RNN, LSTM in Pytorch

- LSTM Network 구성

```python
# 모델 선언
class MyForecastingModel(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(MyForecastingModel, self).__init__()

        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size

        # layer 선언
        self.lstm = nn.LSTM(input_size, hidden_size, batch_first=True)
        self.fc1 = nn.Linear(hidden_size, hidden_size//2)
        self.fc2 = nn.Linear(hidden_size//2, output_size)

    def forward(self, x):
        output, (h_out, c_out) = self.lstm(x)
        h_out = h_out.view(-1, self.hidden_size)
        out = F.relu(self.fc1(h_out))
        out = self.fc2(out)

        return out
```

실습 해볼까요?